

Cross-Origin Resource Sharing (CORS): A Primer

For security reasons, browsers restrict cross-origin HTTP requests. If the coder is not aware of this or forgets about it, it can be pretty frustrating as the required results will not be obtained. Here is an instance of such an occurrence, which the author demonstrates how to solve.



It was a lovely Sunday afternoon in Bengaluru. Unfortunately, I was stuck with my code. I had come up with an idea related to social media, and for the past few weeks, I had been coding to develop a solution for this idea. Well, this article is not to talk about that idea, which is still at a nascent stage and I wouldn't like to share it, just yet. This write-up is to explain why I got stuck with the code and how I finally solved the problem.

First, I will need to give readers a little background. There is a profile-creation service that I have written using Spring Boot, and I have started that service on Port 8082. I have also written an Angular code snippet, through which I am consuming the service to store profile data in MongoDB. The Angular local server is running on Port 4200. A small piece of the code has been reproduced below:

`addProfile service`

```
import com.ikant.repository.Profile;
@RestController
@RequestMapping(value="/api")
public class ProfileServiceController {
    @Autowired
```

```
private ProfileService profileService;
@RequestMapping(method=RequestMethod.POST, value="/
addProfile/profile")
//public String addProfile(@PathVariable String fname)
public ResponseEntity<Profile> addProfile(@RequestBody
Profile profile)
{
    try {
        this.profileService.saveOrUpdate(profile);
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    return new ResponseEntity<Profile>(profile,
HttpStatus.OK);
}
@RequestMapping(method=RequestMethod.GET, value="/
getProfile")
public Profile findName(String fname)
{
    return this.profileService.find(fname);
}
}
```